

2025Year 2nd Semester Syllabus

Created Date	2025-06-30 01:58:39		Last-Modified	2025-09-01 10:49:08	
Course Title	SOFTWARE ENGINEERING		Course Code-Section	CAS3106-01	
Credit/Time/ Experiment,Lab,Pr actical Technique Time	3/Wed3/Fri3,4		Department	School of Computing	
Time	Wed3/Fri3,4		Location	EngHD504/EngHD603	
Exam Date & Time	Midterm exam		Final exam		
Class Language	English		Evaluation Type	Absolute evaluation	

Instructor's Profile	Name	Kim Jieung	Contact Information	Telephone	
	Department	DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING		Mail	JIEUNGKIM@YONSEI.AC.KR
	Office	Formal Computing and AI		Interview information	Wednesday 13:00~15:00

TA's Name & Contact Information	Name	Jonghak Kim, Gijung Im, Juhun Lee, Junyoung Lee	Contact Information	Telephone	
---------------------------------	------	---	---------------------	-----------	--

Course Description Brief Introduction of the Course	This course provides a comprehensive introduction to both the practical and rigorous foundations of software engineering as well as both soft and hard skills. With them, students will learn how to design, develop, test, verify, and maintain high-quality software systems. Topics include software development life cycles, design patterns, testing and coverage, code quality practices such as code reviews and coding style guides, and formal verification techniques.				
	Students will be assessed through hands-on assignments, two individual projects, and two exams. There will be no team projects.				
Course Goals	1.	Korean	소프트웨어 공학에 대한 이해 - 소프트웨어 개발에 공학적 접근이 필요한 이유를 설명할 수 있다. - 소프트웨어 생명주기 전반에 적용되는 주요 기법과 실행 방법을 이해한다.	100%	
		English	Understanding about software engineering - Apply coding, refactoring, and testing techniques to support continuous development and maintainability in real-world projects.		
	2.	Korean	소프트웨어 공학 기법 적용 - 지속적인 개발과 유지 관리를 위한 코드 작성, 리팩토링, 테스트 기법을 실제 프로젝트에 적용할 수 있다.	0%	
		English	Applying software engineering skills - Apply coding, refactoring, and testing techniques to support continuous development and maintainability in real-world projects.		
	3.	Korean	소프트웨어 및 소프트웨어 공학 기법/기술 관련 분석 및 평가 - 주어진 소프트웨어 설계 혹은 코드에 대해 설계 패턴, 테스트 전략, 코드 품질 기준에 따라 분석할 수 있다. - 다양한 개발 방법론이나 설계 대안의 장단점을 평가하고, 특정 상황에 적합한 접근을 판단할 수 있다.	0%	
		English	Analysis and evaluation about software and software engineering methodologies/skills - Analyze given software designs or code using design patterns, testing strategies, and code quality standards. - Evaluate the strengths and limitations of various development methodologies and design alternatives to determine the most suitable approach for a given context.		
				고품질 소프트웨어 및 소프트웨어 공학 기술 개발	

	4.	Korean	- 논리 기반 추론 및 형식 기법을 활용하여 소프트웨어의 정당성을 검증하는 도구나 절차를 설계하고 구현할 수 있다. - 유지보수 가능하고 고품질의 소프트웨어 시스템을 설계하고 구축할 수 있다.			0%
		English	Creating high-quality software and software engineering methodology - Design and implement tools or procedures that verify software correctness using logic-based reasoning and formal methods. - Design and build maintainable, high-quality software systems.			
	5.	Korean				0%
		English				
Core Competencies		The total measurable competencies must be 100%. Each course objective should set the competency as 25%. The core and major competencies should equal at least 50%.				
		협동	50%	프로그래밍능력	40%	문제해결능력
Sub-Competencies/Learning Unit1						
Sub-Competencies/Learning Unit2						
Sub-Competencies/Learning Unit3						
Core Competencies(Liberal Arts)Major competency(		Must reflect the interrelationship between core competencies (elective courses) and major competencies (major studies).				
협동						
Sustainable Development Goals						
Average Recommended Amount of Learning per		Average Reading Volume		Average amount of writing(Based on A4)		
Course Methods (%) Total Amount 100		Lecture	Practice Training	Presentation	Dabate	Team Project
		100%	0%	0%	0%	0%
Course Methods 2 Select Relevant Items		PBL Subject	Capstone Design	CBL, Social Innovation Course	Flipped Classroom	Work Experience, Internsh
Grading Policy(%) Total Amount 100 Free Input for Other Information		Midterm exam	Final exam	Quiz	Individual Assignment	Team Assignment
		30%	35%	0%	30%	0%
Assignment/ Report, Project Guide		Title of Assignment/Project Name, and Method of Filling Out			Submission Deadline	Type of Submission and Method
Prerequisite					Online Course Address	
Course Material	Course Material Name		Author	Publisher	Publish Year	ISBN
주교재	Software engineering		Sommerville, Ian	Pearson Education, Inc	2016 / 10th ed	0133943038

부교재	Software testing and analysis : process, principles, and techniques	Pezzè, Mauro	Wiley	2008	0471455938
-----	---	--------------	-------	------	------------

Main Learner Precautions	A) Course Goal - This course provides a solid foundation in software engineering principles, techniques, and tools. Students will gain essential soft and hard skills to design, develop, test, verify, and maintain high-quality software systems. B) Course Overview a) Key Topics: Throughout this course, students will explore the fundamental concepts, methodologies, and tools used in software development. Key topics include 1-1. The Importance of Software Engineering. 1-1-1. Understand the distinction between a simple program and a software system. 1-1-2. Identify the defining characteristics of software. 1-1-3. Explore the Software Development Life Cycle (SDLC). 1-1-3-1. Understanding each phase: Planning, analysis, design, implementation, testing, deployment, and maintenance. 2. Tools for Assignments 2-1. Version Control & Collaboration: Introduction to Git and GitHub for managing code history and enabling team collaboration. 2-2. Deployment & Maintenance: Introduction to Docker for containerization, ensuring consistency across development, testing, and deployment environments. 3. Soft Skills in Software Engineering 3-1. Design: Understand software design at multiple levels: 3-1-1. Architectural design for high-level structure. 3-1-2. Design patterns for reusable object-oriented solutions. 3-2. Implementation: 3-2-1. Clean code principles and language-specific style guides. 3-2-2. Code review techniques. 3-3. Validation & verification: 3-3-1. Understand the role of validation and verification in the software lifecycle. 3-3-2. Differentiate testing methodologies: White-box vs. Black-box testing 3-3-3. Practical tools 3-3-3-1. Google Test (gtest): Framework for unit testing in C++ 3-3-3-2. GCOV: Tool for measuring code coverage 3-3-3-3. Fuzzing: Intro to automated bug and vulnerability detection 4. Hard Skills in Formal Methods 4-1. Logic & Formal Verification in CS: Foundational logic used to reason about program correctness. 4-2. Formal Design with TLA+: Specification and verification of system behavior using TLA+ models. 4-3. Introduction to model checking and SMT solvers (e.g., Z3) for checking logical consistency and verifying software correctness. b) Practical Experience - The course includes two individual projects and four short assignments designed to experience key concepts and tools from the lectures. By the end of the course, students will have gained the essential skills and knowledge needed to effectively contribute to and manage software development projects. c) Course Organization 1. Weekly Lecture Hours: 3 hours per week 1-1. Lectures will focus on the core content from the lecture notes. 1-2. Additional materials will be provided, including: 1-2-1. Self-study materials: Required for exam preparation. 1-2-2. Supplementary materials: Optional resources for students interested in exploring advanced topics beyond the main lecture content. 2. 2 projects (10% each) and 4 assignments (2.5% each): 30% 3. 2 exams (midterm: 30%, final 35%): 65% 4. Attendance: 5% 4-1. According to the university policy, you will receive an "F" if you miss one-third or more of the total classes 4-2. We primarily check the attendance with the online attendance system 4-3. Please do not use electronic system in the classroom 4-4. If you attend more than two-thirds of the classes and do not violate the rules in the next page, you will receive full attendance points (결석 시에도 출석 점수 감점은 없음) 4-5. Do not share your online attendance code with others 4-6. I will randomly check attendance in person even though we check your attendance with the online code (종종 오프라인으로 출결 확인 예정) 4-7. If any cheating (e.g., sharing the attendance code) is discovered, you will get two penalties (온라인 코드 공유 적발 시 두 가지 감점 적용) 4-7-1. Your attendance will be voided starting from the last confirmed in-person check (마지막 오프라인 출결 확인 시점부터 적발 시점까지 결석 처리) 4-7-2. A portion of your attendance points will be deducted accordingly (출결 점수 감점) 4-8. If cheating is detected more than once, the student who shared the code with you will also receive the same penalty (두 번 이상 적발 시 코드 공유자도 동일 감점 적용) d) Prerequisites
--------------------------	---

	1. Required: CAS1102 (Object oriented programming), CAS2103 (Data structure) 2. You should be familiar with object-oriented languages, including C++. 2-1. You should be able to learn different kinds of languages, including C++, Python, etc. 2-2. You should be familiar with implementing data structures as well as be able to analysis pros and cons of several data structures. 3. Required: you should have a Github account. 4. Recommended: CAS2106 (Introduction to Computer Science), CAS2107 (Computer System), CAS3109 (Automataform Language) e) Textbooks - There are no textbooks that cover all lecture topics. However, the following textbook is good for studying the overall process of software engineering. 1. "Software Engineering", 10th Edition by Ian Sommerville 2. Sub-textbooks and Supplementary Materials - The following list is a useful but non-exhaustive list of items containing related materials from our lectures. Several materials are available online, so you can access them for free. 2-1. Sub-textbooks 2-1-1. "System Design Interview: An Insider's Guide", Second Edition, Alex Xu 2-1-2. "Design patterns" (https://refactoring.guru/design-patterns) 2-1-3. "Software Engineering at Google: Lessons Learned from Programming Over Time", 1st Edition by Titus Winters (Author), Tom Manshreck (Author), Hyrum Wright (Author) 2-1-4. "The Fuzzing Book, Tools and Techniques for Generating Software Tests", by Andreas Zeller, Rahul Gopinath, Marcel Böhm, Gordon Fraser, and Christian Holler (https://www.fuzzingbook.org/) 2-2. Supplementary materials (tools) 2-2-1. Docker: https://www.docker.com/ 2-2-2. Github docs: https://docs.github.com/en 2-2-3. Google Styleguide: https://google.github.io/styleguide/ 2-2-4. AFL++: https://aflplusplus.com/ 2-2-5. OSS Fuzz: https://github.com/google/oss-fuzz 2-2-6. GoogleTest: https://github.com/google/googletest 2-2-7. GCOV: https://gcc.gnu.org/onlinedocs/gcc/Gcov.html 2-2-8. UPPAAL: https://uppaal.org/ 2-2-9. TLA+: https://lamport.azurewebsites.net/tla/tla.html 2-2-10. CBMC: https://www.cprover.org/cbmc/ 2-2-11. Z3: https://github.com/Z3Prover/z3 C) Contact information: 1. In case of sending an email: To ensure your inquiries are not missed, please avoid using the email address listed in this document. Instead, use "fcai.yonsei@gmail.com" with the subject line containing the [CAS3106-01-00] tag, enabling both the primary instructor and TAs to address your questions together. 2. You may also email "jjeungkim@yonsei.ac.kr" with any questions. However, please note that inquiries related to this course may be shared with the TAs. D) Exam 1. Midterm: October 24th (Friday) 11:10~12:50 (100 mins) 1. Final: December 19th (Friday) 11:10~12:50 (100 mins) 3. Exams will be conducted as closed-book assessments; however, you are allowed to prepare a single-page (A4, double-sided) cheat sheet. F) Misc: - The detailed schedules for lectures, assignments, and projects are subject to minor changes, and any changes will be announced in advance.
Attatchment	

Weekly Plan

week	Period	Weekly Topic & Contents	Remarks
1		Introduction - Course overview, syllabus - What is software, software engineering motivation	(9.1.) Fall semester classes begin (9.3. - 9.5.) Course add and drop period

2		Software development processes & Basic Tools - SDLC and process models - Version control, container, license - Tools: GitHub actions, Docker Assignment #1: Github actions & Docker (Due on week 5)	
3		Design & OOP principles - System architecture design - Case study (e.g., chatbot, Instagram, etc.) - OOP design principles	
4		Design - Design pattern overview - Case study (e.g., factory method, abstract factory, builder, singleton, composite, decorate, façade, visitor, strategy, observer, chain of responsibility, etc.)	
5		Implementation - Clean code, style guides, code review - Tools: Code review via GitHub PRs Testing - Testing overview - White-box & black-box testing Assignment #2: Techniques on implementation (Due on week 6) Note: - No class on Friday due to the holiday - Friday lecture will be provided online	(10.3.) National Foundation Day
6		Testing - Coverage criteria - Tools: GoogleTest, Python unittest, GCOV - Advanced testing techniques Project #1: Extend a software testing tool (Due on week 9) Note: - No class on Wednesday due to the holiday - Wednesday lecture will be provided online	(10.5. - 10.7.) Chuseok Holidays (10.8.) First third of the semester ends (10.9.) Hangul Proclamation Day
7		Testing - Advanced testing tools: AFL++ , OSS-Fuzz Static analysis - Introduction to static analysis and tools Note: - No class on Friday due to a scheduled business trip - Friday lecture will be provided online	(10.13. - 10.15.) Course withdrawal period
8		Midterm exam (Covers weeks 1-7)	(10.20. - 10.26.) Midterm Examinations
9		Formal methods - Introduction to formal methods - Logic in CS: Propositional and predicate logic, linear temporal logic, computation tree logic Assignment #3: Software testing & Logic (Due on week 11)	(10.27. - 10.29.) Application Period for S/U evaluation
10		Model Checking - Model checking - Tools: UPPAAL and TLA+ - Case study (e.g., railway timetables, smart contracts, etc.) Note: - No class on Friday due to a scheduled workshop (forum) - Friday lecture will be provided online	
11		SAT & SMT solving - Boolean Satisfiability (SAT) problem - DPLL Assignment #4: Formal specifications & model checking (Due on week 13)	(11.13.) Second third of the semester ends
12		SAT & SMT solving - From SAT to SMT - Tools: Z3	
13		Source code verification - Programming language semantics - Hoare logic - Tools: CBMC, Z3 (revisit) Project #2: Verify a program (Due on early in week 17)	

14		More than automated formal verification - Abstract interpretation - Deductive reasoning with interactive theorem provers	
15		Note: No class (reading periods)	(12.8. - 12.14.) Self-study
16		Final (Covers Weeks 9-14)	(12.15. - 12.21.) Final Examinations

계절제 수업계획서 작성 시 이전 계절제 수업계획서를 복사하시거나, 직접 입력하시기 바랍니다.

정규학기 수업계획서를 복사하는 경우 오류가 발생할 수 있습니다.

• Students with disabilities(SWDs) can request accommodations related to lectures, assignments, or tests by contacting the course professor at the beginning of semester.
(However, accommodations may vary depending on the essentiality of lecture and discretion of professors.)

[Lecture]

- Visual Impairment: alternative, braille, enlarged reading materials, note-taker
- Physical Impairment: alternative reading materials, access to classroom, note-taker, assigned seat
- Hearing Impairment: note-taker/stenographer, recording lecture
- Intellectual Disability/Autism: note-taker

[Assignments and Test]

- Visual/Physical/Hearing Impairment: (reasonable) extra days for submission, alternative type of assignment, extended test time, alternative type of test, arranging separate test room, and proctors, test ghostwriter
- Intellectual Disability/Autism: (reasonable) extra days for submission, alternative type of assignment